

C++第二章

1.C++运算符

- 分类：
 - 移位 / 重定向运算符：<< 和 >>
 - 赋值与复合赋值：=, +=, -=, *=, /= 等
 - 算术运算符：+, -, *, /, %, ++, --
 - 逻辑运算符：!, &&, ||, <=, >=, >, <, ==, !=
 - 条件运算符（三元运算符）：cond ? v1 : v2
- Shift / redirect
 - 对整数 (int) 使用时<< 和 >> 表示二进制位移。
 - 例如：x = x >> 1 相当于 x = x / 2（右移 1 位除以 2，左移 1 位乘以 2）。
 - 对对象 (object) 使用时<< 和 >> 表示数据流输入/输出，即重定向/传输数据。
 - 例如 cout << "Hello World" << '\n'; 用 << 把内容写到输出流 cout。
 - cin 则是输入流，对应用 >> 从键盘读取数据。
 - 输入的两种方式：
 - cin >> x：读一个数字或字符串，遇到空格/换行停止。
 - getline(cin, s)：整行输入，包含空格。

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main() {
6     string s;
7     cout << "请输入一句话: ";
8     getline(cin, s); // 读取一整行
9     cout << "你输入的是: " << s << endl;
10    return 0;
11 }
```

示例输入

```
1 I love C++ programming!
```

输出

```
1 你输入的是: I love C++ programming!
```

和 cin >> s 的区别

- cin >> s

- 读取一个 **单词**，以 **空格、Tab、换行** 作为分隔符。
- 输入 "Hello World" → `s = "Hello"`，只读到第一个单词。
- `getline(cin, s)`
 - 读取 **一整行**，包括空格。
 - 输入 "Hello World" → `s = "Hello world"`，整行读入。
 - `getline` 会读整行，直到换行符。
 - `cin` 会读到空格或换行为止，剩余内容留在缓冲区。
 - 可以用 `cin.fail()` 来检测输入是否有效。

默认情况下，`getline` 遇到换行符结束读取。你也可以 **指定分隔符**，例如：

```
1 getline(cin, s, ','); // 读到逗号 ',' 停止
```

输入：

```
1 apple,banana,orange
```

第一次调用得到 `s = "apple"`，第二次调用得到 `s = "banana"`。

- 示例代码：

```
1 int x{};
2 cin >> x;
3 string dummy;
4 getline(cin, dummy); // 接收输入行中剩余的无效内容
5
6 if (cin.fail()) {
7     cout << "Unknown input for x\n";
8     return 0;
9 }
```

- `cin >> x` 读取整数。
- 如果用户输入了 **部分有效 + 部分无效** 的数据，比如 "123abc"，那么 123 会被存进 `x`，剩下的 "abc" 会被 `getline(cin, dummy)` 接收。
- 如果输入完全无效（例如 "abc"），就会触发 `cin.fail()`。

这样做的目的：把不合规的输入清理掉，避免后续逻辑出错。

- Assignment operator
 - `=` 是赋值运算符，`==` 才是比较运算符。
- 6 赋值运算符本身**有返回值**，即被赋的值。

示例：

```
1 int x{};
2 int y{};
3
4 cout << (x = 100) << '\n'; // 输出 100
5 cout << (x = x + 100) << '\n'; // 输出 200
6 x += 100; // 推荐这种写法
```

- `y = x = 100`; 会先执行 `x = 100`, 返回 100, 然后赋给 `y`, 结果 `x` 和 `y` 都是 100。

How C++ gets raw performance

这里讲的是 **C++ 和 Python 的赋值机制差异**:

- 在 Python 里, 变量其实是对象的引用, 赋值后变量 `id` (内存地址) 会变:

```
1 >>> a = 42
2 >>> id(a) # 比如 140351051290024
3 >>> a = a + 1
4 >>> id(a) # 地址变了
```

- 每次重新赋值, Python 可能会创建一个新的对象。

而在 C++ 中:

- 变量就是一块固定的内存, 赋值只是覆盖旧值, 不会新建对象。
- 这就是 **C++ 原始性能高的原因之一**: 少了额外的对象分配和引用管理。

Explanation

- **C++ 与 Python 的赋值机制对比**:

- 在 C++ 里, 赋值通常会复用同一块内存地址, 性能更高, 更可控。
- 在 Python 中, 每次赋值可能涉及内存分配和数据复制 → 内存消耗更大。
- Python 还会有垃圾回收 (GC), 在回收过程中可能导致程序暂停 → 存在不确定性。

- 图示说明: Python 赋值时可能会频繁分配和标记内存以供删除。

Python tends to optimize for productivity, not for performance

- Python 库 (如 Pandas) 通常为了 **开发效率** 优化, 而非运行性能。
- 示例展示了 Pandas DataFrame 修改时会分配新内存:

```
1 a = pd.DataFrame({'d': [1, 2, 3]})
2 b = a
3 id(a) == id(b) # True, 两个变量指向同一对象
4 id(a.query('d==1')) != id(a) # 新查询返回了新对象, 占用新内存
```

- C++ 的开发者对内存分配/释放更有把控, 因此性能更可预测。

• Arithmetic operators - increment/decrement

- 除了常见的算术运算符 (`+` `-` `*` `/` `%`), 这里重点讲 **自增自减运算符**:
 - `x++` (后置递增): 返回递增前的值, 再进行递增。
 - `++x` (前置递增): 先递增, 再返回递增后的值。
 - 类似的规则适用于 `x--` 和 `--x`。

例子:

```
1 // ++x
2 int pre_plus(int x) { return x + 1; }
3
4 // x++
5 int post_plus(int x) { int y = x; x += 1; return y; }
```

Difference between ++x and x++

- 两者区别:
 - ++x 不需要保存临时值, 更高效。
 - x++ 需要一个临时变量保存原值 → 稍慢。
- 例子:

```
1 int x = 1;
2 int y = ++x; // x=2, y=2
3
4 x = 1;
5 int y = x++; // x=2, y=1
```

- 输出示例:

```
1 cout << x++ << ", " << ++x; // 结果: 2, 4
```

- 现代编译器会对无临时变量需求的情况进行优化, 但在 `y = x++` 这种情况下, 依然会用临时变量。

• Logical C++ operators

- `!`: 逻辑非, `true` → `false`, `false` → `true`。
- `A && B`: 逻辑与, 只有当 A 和 B 都为 `true` 时, 结果才是 `true`。
- `A || B`: 逻辑或, 只要 A 或 B 有一个为 `true`, 结果就为 `true` (包括两者都 `true` 的情况)。
- 短路求值 (short-circuit evaluation):
 - 如果 A 为 `false`, 表达式 `A && B` 直接为 `false`, 不再计算 B。
 - 如果 A 为 `true`, 表达式 `A || B` 直接为 `true`, 不再计算 B。

&& (logical and) and || (logical or)

- 运算优先级:
 - `!` 优先级最高。
 - `&&` 的优先级高于 `||`。
 - 但实际编程时推荐用括号 `()` 来消除歧义。
- 示例:

```
1 ((1<=1) || (0==0)) && ((5<5) || (5!=5)) // false
2 ((1<=1) || (0==0)) && ((5<5) || (5==5)) // true
```

- 括号非常重要, 容易出错。

Explanation

- 在处理多个条件时, 可以先用 **bool 类型的中间变量**, 让代码更清晰。
例如:

```

1 bool is_later_date = date1 > date2;
2 bool is_equal_balance = bal_pre == bal_post;
3
4 if (is_later_date && is_equal_balance) {
5     // do something
6 }

```

- 这样比直接写 `(date1 > date2) && (bal_pre == bal_post)` 更直观、可读性更高。

Usage of &&, || and !

- 逻辑运算符 **&& (与)**、**|| (或)**、**! (非)** 在控制结构 (如 `if`, `while`, `for`) 中非常常见。
- 在 C++ 中, `0` 表示 `false`, 非零值表示 `true`。但推荐使用 `true/false` 代替整数, 提高代码可读性。
- 对 `double` 类型要小心, 比如 `0.0` 是 `false`, 而任何非零 `double` 值都是真。但是两个非常接近的 `double` 值可能存在 `1e-9` 的差异, 因此逻辑判断最好用 `bool`。
- 示例:

```

1 if (-1) { // -1 也是 true
2     cout << "-1" << '\n';
3 }
4
5 if (cin.fail()) {
6     cout << "failed" << '\n';
7 }

```

Usage of &&, || and !- examples

- 展示了 **短路求值** 的代码习惯 (idioms)。
- 两种写法等价, 可以帮助记忆:

```

1 cin.fail() && cout << "failed" << '\n';
2 // 读作 "fail and cout"
3
4 !cin.fail() || cout << "failed" << '\n';
5 // 读作 "not fail or cout"

```

- 这种方式利用了 **&&** 和 **||** 的短路性质来简化代码。
- **Conditional ternary operator**
 - 介绍了条件运算符 `?:`, 它是 `if-else` 的简写形式, 用来给变量赋值。
 - 一般形式:

```

1 condition ? value_if_true : value_if_false;

```

- 示例:


```
1 double result = j < k ? sin(theta) : cos(theta);
2
3 // 等价于
4 if (j < k) {
5     result = sin(theta);
6 } else {
7     result = cos(theta);
8 }
```

2. Control Structures

- **控制结构 (Control Structures)** 是程序设计的基础，决定了程序的执行流程。

常见三类：

1. **顺序 (Sequence)**：代码从上到下依次执行。
2. **条件 (Conditional)**：根据条件执行不同的代码块，例如 `if-else`。
3. **循环 (Iteration)**：重复执行代码块，例如 `for`, `while`。

C++ 中的用法：

1. **条件执行**：`if-else`, `switch`。
2. **循环执行**：`for`, `while`, `do-while`。
3. **跳转语句**：`break`, `continue`, `goto` (推荐避免使用 `goto`)。

- **如何设计条件流程块**，让程序逻辑更清晰、简单：

1. **让条件简单且容易理解** (避免复杂的逻辑表达式)。
2. **把高优先级的条件放在前面** (提高效率和可读性)。
3. **减少嵌套层级** (降低复杂度)。
4. **设置防护条件 (guards)**，避免无效或错误的情况进入代码块。

- **Make the Conditions Clear for Easy Review**

当条件表达式过于复杂时，应该如何简化：

1. **考虑反面条件**：例如 `not A or not B` 等价于 `not (A and B)`。
2. **把条件分解为更小的部分**，逐步构建复杂逻辑。
3. **使用有意义的名字存储单个条件**，组合成最终逻辑。

- **Nested if-else**

- 嵌套层数越多，代码复杂度就越高。执行路径的数量 = 每层分支数的乘积，测试会变得很难。
- 目标：**减少嵌套层数**，避免在 `if-else` 里写很长的代码。

优化方法：

1. **简化条件** (用中间变量或函数)。
2. **优先处理简单情况**，用 `return` 或 `break` 提前结束。
3. **大块代码提取成函数**，让 `if-else` 更简洁。

- **Example for Complex if-else - 1**

- 当 `if-else` 条件很长时，代码的可读性会变差。
- **优化方法**：
 - 如果分支里最终会 `return`，可以把逻辑提取成函数，提前返回，简化结构。

- 这样一眼就能看出每个条件对应的处理，流程更清晰。

```
1  if (cond1) {  
2      if (cond2) {  
3          // 很多行代码  
4      } else {  
5          // 很多行代码  
6      }  
7  } else {  
8      if (cond3) {  
9          // 很多行代码  
10     }  
11 }  
12 return;
```

简化写法：

```
1  if (cond1) { task1(); return; }  
2  if (cond2) { task2(); return; }  
3  if (cond3) { task3(); return; }
```

- 当错误发生的概率很低时，处理逻辑可以放在最后。
- 优化思路：**避免层层嵌套的 if-else**，直接用错误码简化逻辑。

```
1  if (check_error_A()) {  
2      handle_errorA();  
3  } else if (check_error_B()) {  
4      handle_errorB();  
5  } else if (check_error_C()) {  
6      handle_errorC();  
7  } else {  
8      send_order_to_exchange();  
9  }
```

优化后：

```
1  int error_type = check_errors();  
2  if (error_type == 0) {  
3      send_order_to_exchange();  
4  } else {  
5      handle_error(error_type);  
6  }
```

- **Set Guard with Exception**

- **Guard (守护)**：在代码中加入检查条件，当遇到非法情况时，立即抛出异常（而不是继续执行）。
- 通常的做法是 **停止程序并提示清晰的错误信息**。
- 在 C++ 中：
 - `throw` 抛出异常
 - `#include <stdexcept>` 提供 `logic_error`（逻辑错误）和 `runtime_error`（运行时错误）

- 一般用 `logic_error`

- 示例:

```
1 #include <stdexcept>
2 #include <sstream>
3
4 if (x < 100) {
5     ...
6 } else if (x < 50) { // 逻辑错误
7     ...
8 } else {
9     ostringstream ss;
10    ss << "should not reach here as x in [0, 100), x=" << x;
11    throw logic_error(ss.str());
12 }
```

当程序运行到不应该到达的地方时，会抛出异常，避免错误继续扩散。

Guards Can Be Set Up Generally

- **Guard (守护条件)** 是一种代码防护手段，可以避免程序进入非法或不合理的状态。
- 常见方法:
 - `assert`: 只在 **debug 模式** 下运行，release 模式下会被移除。
 - `throw logic_error(...)`: 无论在 debug 还是 release 模式下都能运行，并且可以抛出清晰的错误信息。
- 对比:
 - `assert` 停止程序，但信息有限。
 - `throw logic_error` 可以提供更详细的错误信息。
- **编译模式:**
 - Debug Mode: 带有额外调试信息（变量名、代码位置）。
 - Release Mode: 去掉调试信息，优化运行速度和体积。

展示了如何用 **guard + 异常** 来保护函数的输入。

- 示例:

```
1 void calculate_interest(double interest_rate) {
2     // 只允许正利率
3     if (interest_rate < 0)
4         throw logic_error("interest_rate must be positive");
5
6     // 或者用三元表达式简化写法
7     interest_rate < 0 ?
8     throw logic_error("interest_rate must be positive") :
9     true;
10 }
```

这样在传入非法参数时，程序会立刻抛出异常，避免继续运行。

- **Switch Statement**

- **switch** 是一种条件分支语句，只能用于 **整数类型 (含 char)** 和 **enum 枚举类型**。
- 它的作用类似于一系列的 `if-else`，但更清晰。

◦ 注意:

- 每个分支用 `case` 标记。
- `case` 结束要用 `break`, 否则会发生“贯穿”(fall-through)。

🔴 示例:

```
1 int input;
2 cout << "1. Play game\n"
3   << "2. Load game\n"
4   << "3. Play multiplayer\n"
5   << "4. Exit\n"
6   << "Selection: ";
7 cin >> input;
8
9 switch (input) {
10     case 1: playgame(); break;
11     case 2: loadgame(); break;
12     case 3: playmultiplayer(); break;
13     case 4: cout << "Thank you for playing!\n"; break;
14     default: cout << "Error, bad input, quitting\n"; break;
15 }
16 cin.get();
```

总结:

1. **Guard** 用于保护函数和逻辑, 防止非法输入 (`assert` vs `throw`)。
2. **switch** 是 `if-else` 的替代写法, 适合整数/枚举条件分支, 代码更简洁。

Allow Fall-Through

- 在 `switch` 语句中, 如果 **没有写 `break`**, 会产生 **fall-through (贯穿)**。
- 这种情况有时是我们希望的, 比如多个 `case` 共享相同的执行逻辑。

示例:

```
1 switch (input) {
2     case 1:  // 没有 break, 会继续执行 case 2
3     case 2:
4     ...
5     case 9:
6         load_digit(input);
7         break;
8     default:
9         cout << "Error, bad input, quitting\n";
10        break;
11 }
```

如果 `input` 是 1 或 2 到 9, 都会执行 `load_digit(input)`。

• Loop (循环)

- **循环 (loop)** 用于重复执行代码块。
- 如果提前知道循环次数 → 用 `for` 循环。
- 如果事先不知道次数 → 用 `while` 循环。

- 每次循环执行称为 **一次迭代 (iteration)**。
- 为了避免 **死循环**，必须设置明确的退出条件。

(1) for-loop

- 基本语法：

```
1 for (initialization; condition; update) {  
2     statements;  
3 }
```

- 特点：

1. **执行条件**：只要 `condition` 为真，就会继续循环。
2. **作用域规则 (scope rule)**：在 `for` 内部声明的变量只在循环中有效。
3. 可以嵌套 `for` 循环，用于处理多维数据结构。

Type (I) for-Loop

- **执行流程**：

1. 初始化 (`init`)
2. 检查条件 (`cond`)
3. 执行循环体 (`block`)
4. 更新 (`update`)
5. 返回条件检查 (`goto 2`)

- 条件在进入循环前检查，所以如果一开始条件不成立，循环可能 **一次都不会执行**。

示例：

```
1 for (int i = 0; i < 5; i++) {  
2     cout << i << " ";  
3 }  
4 // 输出: 0 1 2 3 4
```

for-Loop Example - 1

- **无限循环**：`for(;;)` 会一直循环。
- `for` 循环中的变量声明 (如 `auto i = 0`) 是 **迭代变量 (iterator variable)**。
- 推荐用 `++i`，虽然 `i++` 和 `++i` 在循环里没有区别 (编译器会优化)。
- 示例：

- **错误写法**：忘记初始化 `i`。

```
1 for (size_t i; i < 10; ++i) // 错误, i 未初始化  
2     cout << i << '\n';
```

- **正确写法**：

```
1 for (auto i = 0; i < 10; ++i)  
2     cout << i << '\n';
```

- 注意：循环外不能再用 `i`，因为作用域结束。

Use of `size_t`

- 推荐在循环变量中使用 `size_t` 而不是 `int`，因为：
 - `size_t` 是 **无符号整型**，表示系统能处理的最大对象大小。
 - 常用作数组下标或内存计数，更符合语义。
 - 示例：

```
1 for (size_t i = 0; i < vec.size(); ++i) {  
2     cout << vec[i] << '\n';  
3 }
```

- 如果不确定类型，也可以用 `auto`。

for-Loop Example 2

- 说明 **循环变量作用域** 和 **不修改计数器的规则**。
- 错误示例：

```
1 for (size_t i = 0, temp_sum = 0; i < 10; ++i) {  
2     temp_sum += i;  
3     if (temp_sum > 30) break;  
4 }  
5 cout << temp_sum; // 错误, temp_sum 超出作用域
```

- 正确示例：
 - 把需要保留的变量放在循环外声明。
 - 不要在循环中随意修改循环计数器。

```
1 for (size_t i = 0; i < 10; ++i) {  
2     for (size_t j = i+1; j < 10; ++j) {  
3         // 正确用法, i 和 j 独立  
4     }  
5 }
```

for-Loop Example 3

- 推荐：变量声明尽量放在 **最小作用域**，避免外部变量被修改。
- 不推荐写法：

```
1 int i{0}, x{0};  
2 for (i = 0; i < N; ++i) {  
3     x = value * 5;  
4     sum += x;  
5 }
```

- 推荐写法：

```

1  for (size_t i = 0; i < N; ++i) {
2      int x = value * 5; // 局部变量，作用域仅限循环体
3      sum += x;
4  }

```

字符串拼接的推荐写法:

```

1  std::string str("prefix_");
2  for (size_t i = 0; i < N; ++i) {
3      std::cout << str + value[i];
4  }

```

✎ 把不变的 `str` 声明在循环外，只在循环中处理变化的部分。

while 循环:

```

1  while (condition) {
2      statement;
3  }

```

- 在条件为真时执行循环体。
- 要小心无限循环，确保条件在有限次数后变为假。

while-Loop: Example 1

题目：不断将 3072 除以 2，直到结果为奇数，输出最后的结果。

代码:

```

1  int x{3072};
2  while (x % 2 == 0) { // 当 x 可以被 2 整除时继续循环
3      x /= 2;         // 除以 2
4  }
5  cout << x << '\n'; // 输出奇数结果

```

思路:

- 每次循环判断是否能被 2 整除，能则继续除。
- 直到结果为奇数（不能被 2 整除），输出最后值。
- 保证最终结果要么是 **奇数**，要么是 **1**。

do-while 循环:

```

1  do {
2      statement;
3  } while (condition);

```

- 至少会执行一次循环体，然后再检查条件。
- 与 `while`、`for` 的区别：`do-while` 保证执行 ≥ 1 次。

continue (流程图)

- `continue` 的作用：跳过当前循环剩余的语句，直接进入下一次循环的条件检查。
- 常用于 **跳过某些情况**，例如过滤偶数，只处理奇数。

continue 示例

```

1 for (size_t i = 0; i < 10; ++i) {
2     if (i % 2 == 0)    // 如果是偶数
3         continue;    // 跳过这次循环，不执行下面的语句
4     cout << i << '\n'; // 只会输出奇数
5 }

```

输出结果: 1 3 5 7 9

- 无限循环示例:

```

1 do {
2     continue;
3 } while (true); // 死循环，永远不会退出

```

continue: Example 1

- 示例: 输出 1 到 100 之间 **不能被 13 整除** 的数。

```

1 for (size_t i = 1; i <= 100; ++i) {
2     if (i % 13 == 0) {
3         continue;    // 跳过能被 13 整除的数
4     }
5     cout << i << '\n'; // 输出其他数
6 }

```

continue 的作用: 跳过本次循环, 继续下一次循环。

- break

- 在循环体中, 如果条件满足, 就会直接跳出整个循环, 而不是仅仅跳过一次循环。

break 示例

- 示例: 找到第一个能同时被 13 和 7 整除的数。

```

1 size_t i = 0;
2 while (++i, i < 100) {
3     if (i % 13 == 0 && i % 7 == 0) {
4         break;    // 找到后立即退出循环
5     }
6 }

```

break 的作用: 提前终止循环, 不再继续执行。

- while vs for

- while 循环: 通常基于条件控制, 适合 **不知道循环次数** 的情况。

- for 循环: 常用于已知循环次数的情况, 或有明确的迭代变量。

- 示例场景:

- 给定 `vector<double>`, 从第一个元素开始累加, 直到和 ≤ 1 为止。

- 用 while 更自然, 因为条件是“总和 ≤ 1 ”, 而不是固定次数。

- while 示例


```
1 vector<double> items{0.4, 0.5, 0.6};
2 double sum{0};
3 int i{0};
4 while (true) {
5     const auto sum2{sum + items[i]};
6     if (sum2 > 1) { break; }
7     sum = sum2; ++i;
8 }
```

while 在循环中 **依赖条件退出**，但如果数据不可解（如数组太短），会导致无限循环或越界。

◦ **for 示例**

```
1 vector<double> items{0.4, 0.5, 0.6};
2 double sum{0};
3 for (auto i = 0; i < items.size(); ++i) {
4     const auto sum2{sum + items[i]};
5     if (sum2 > 1) { break; }
6     sum = sum2;
7 }
```

for 循环更自然，因为它本身有边界限制，更适合处理这种“有限次数”的情况。

• **Tracing/Debug Techniques**

1. **打印调试 (print debug)**

- 使用全局常量控制是否打印调试信息：

```
1 const auto DEBUG = true;
2 DEBUG && cout << "var=" << variable << '\n';
```

方便在调试结束后关闭打印。

2. **使用调试器 (debugger)**

- 设置断点，运行时停在断点位置，可以逐步检查变量值和执行流程。